

SQL Assignment & Practice Bank — 200 Questions

1. Database & Table Creation (1–20)

-- Q1. Create a database named company_db.

```
create database company_db;
```

-- Q2. Select and use the company_db database.

```
use company_db;
```

-- Q3. Create an employees table with employee_id, employee_name, department, salary, city, and hire_date columns.

-- Q4. Make employee_id the PRIMARY KEY.

-- Q5. Make employee_name NOT NULL.

-- Q6. Add a UNIQUE constraint to an email column.

-- Q7. Add a CHECK constraint so salary must be greater than 0.

-- Q8. Add a DEFAULT value of 'Hyderabad' for the city column.

```
create table employeee (  
  emp_id int primary key,  
  emp_name varchar(30) not null,  
  department varchar(20),  
  salary int check(salary > 0),  
  email varchar(100) unique,  
  city varchar(30) default 'hyderabad',  
  hire_date varchar(10)  
);
```

-- Q9. Create a departments table with department_id, department_name, location, and budget.

```
create table department(  
  dept_id int,  
  dept_name varchar(30),  
  location varchar(40),  
  budget int  
);
```

-- Q10. Make department_id the PRIMARY KEY in the departments table.

```
alter table department
add primary key (dept_id);
```

-- Q11. Create a projects table with project_id, project_name, department_id, and budget.

```
create table project(
project_id int,
project_name varchar(30),
dept_id int,
budget int
);
```

-- Q12. Add a FOREIGN KEY from employees.department_id to departments.department_id.

```
alter table employeee
add column dept_id int;
```

```
alter table employeee
add foreign key (dept_id)
references department(dept_id);
```

-- Q13. Insert one employee record.

```
insert into employeee
(emp_id,emp_name,department,salary,email,city,hire_date,dept_id) values
(1,'raj nandani','data
science',50000,'raj.nandani1117@gmail.com','jharkhand','2024-11-17',200);
```

-- Q14. Insert five employee records in a single INSERT statement.

```
insert into employeee
(emp_id,emp_name,department,salary,email,city,hire_date,dept_id) values
(2,'raj ','data science',60000,'raj@gmail.com','delhi','2023-12-17',100),
(3,'rahul','data analysis',10000,'rahul@gmail.com','mumbai','2021-03-11',300),
(4,'raj kumar ','data ',80000,'rajkumar@gmail.com','delhi','2024-12-17',100),
(5,'dheeraj ','CS',800000,'dk@gmail.com','kerla','2025-11-06',400),
(6,'avi ','science',90000,'avi@gmail.com','bhopal','2019-12-17',200);
```

-- Q15. Insert a department record.

```
insert into department
(dept_id,dept_name,location,budget) values
(100,'data science','bbsr',30000),
```

```
(200,'data ','delhi',40000),  
(300,'data science','bbsr',30000),  
(400,'data analysis','bhopal',20000),  
(500,'data science','bbsr',60000),  
(600,'data ','jharkhand',670000);
```

-- Q16. Create a students table with PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.

```
create table students(  
stu_id int primary key,  
stu_name varchar(30) not null,  
email varchar(100) unique,  
age int check(age<=18),  
address varchar(30) default 'delhi'  
);
```

-- Q17. Create a table named attendance with employee_id, attendance_date, and status.

```
create table attendance(  
emp_id int,  
attd_date varchar(20),  
status varchar(10)  
);
```

-- Q18. Create a table only if it does not already exist.

```
create table temp_data(  
id int,  
name varchar(30)  
);
```

-- Q19. Describe or display the structure of the employees table.

```
desc employee;
```

-- Q20. Drop a temporary table named temp_data if it exists.

```
drop table temp_data;
```

2. Basic SELECT & Filtering (21–45)

-- Q21. Display all columns from the employees table.

```
select * from employee;
```

-- Q22. Display only employee_name and salary.

```
select emp_name , salary from employeee;
```

-- Q23. Display employees who live in Hyderabad.

```
select * from employeee  
where city = 'hyderabad';
```

-- Q24. Display employees whose salary is greater than 60000.

```
select * from employeee  
where salary > 60000;
```

-- Q25. Display employees whose salary is less than or equal to 50000.

```
select * from employeee  
where salary <= 50000;
```

-- Q26. Display employees whose salary is between 50000 and 70000.

```
select * from employeee  
where salary between 50000 and 70000;
```

-- Q27. Display employees who live in Hyderabad, Bangalore, or Mumbai using IN.

```
select * from employeee  
where city in('hyderabad','bangalore','mumbai');
```

-- Q28. Display employees who do not live in Hyderabad.

```
select * from employeee  
where city != 'hyderabad';
```

-- Q29. Display employees whose names start with 'A'.

```
select * from employeee  
where emp_name like 'a%';
```

-- Q30. Display employees whose names end with 'a'.

```
select * from employeee  
where emp_name like '%a';
```

-- Q31. Display employees whose names contain the letter 'i'.

```
select * from employeee  
where emp_name like '%i%';
```

-- Q32. Display employees whose department is NULL.

```
select * from employeee  
where department is null;
```

-- Q33. Display employees whose department is NOT NULL.

```
select * from employeee  
where department is not null;
```

-- Q34. Display employees hired after 2022-01-01.

```
select * from employeee  
where hire_date > '2022-01-01';
```

-- Q35. Display employees hired between two given dates.

```
select * from employeee  
where hire_date between 2023-12-11 and 2024-12-11;
```

-- Q36. Display employees with salaries greater than 50000 AND who live in Hyderabad.

```
select * from employeee  
where salary > 50000 and city = 'hyderabad';
```

-- Q37. Display employees who live in Hyderabad OR earn more than 70000.

```
select * from employeee  
where salary > 70000 and city = 'hyderabad';
```

-- Q38. Display employees who are NOT in the HR department.

```
select * from employeee  
where department not in('hr');
```

-- Q39. Display distinct cities from the employees table.

```
select distinct(city) from employeee;
```

-- Q40. Display distinct departments from the employees table.

```
select distinct(department) from employeee;
```

-- Q41. Display employees sorted by salary in ascending order.

```
select * from employeee  
order by salary;
```

-- Q42. Display employees sorted by salary in descending order.

```
select * from employeee  
order by salary desc;
```

-- Q43. Display employees sorted first by department and then by salary descending.

```
select * from employeee  
order by (department) asc, (salary) desc;
```

-- Q44. Display the first 5 employees after sorting by employee_id.

```
select * from employeee  
order by emp_id  
limit 5;
```

-- Q45. Display the top 3 highest-paid employees.

```
select * from employeee  
order by salary desc  
limit 3;
```

3. UPDATE, DELETE, ALTER & Basic Functions (46–65)

-- Q46. Update the salary of one employee using employee_id.

```
update employeee  
set salary = 1000000  
where emp_id = 3;
```

-- Q47. Increase the salary of all employees by 10 percent.

```
SET SQL_SAFE_UPDATES = 0;  
update employeee  
set salary = salary * 1.10 ;  
select * from employeee;
```

-- Q48. Increase the salary of employees in the Data Science department by 5000.

```
update employeee  
set salary = salary+ 5000  
where department = 'data science';
```

-- Q49. Change the city of all employees currently living in Bangalore to Bengaluru.

```
update employeee  
set city = 'bengaluru'  
where city = 'bangalore';
```

-- Q50. Delete one employee using employee_id.

```
delete from employeee  
where emp_id = 4;
```

-- Q51. Delete employees whose salary is below a specified amount.

```
delete from employeee  
where salary < 55000;
```

-- Q52. Add an email column to the employees table.

```
alter table employeee  
add emails varchar(30);
```

-- Q53. Add a phone_number column to the employees table.

```
alter table employeee  
add phone_no bigint;
```

-- Q54. Modify the length of the employee_name column.

```
alter table employeee  
modify emp_name varchar(50);
```

-- Q55. Rename the city column to employee_city.

```
alter table employeee  
rename column city to emp_city ;
```

-- Q56. Drop the phone_number column.

```
alter table employeee  
drop column phone_no;
```

-- Q57. Display employee names in uppercase.

```
select upper(emp_name) from employeee;
```

-- Q58. Display employee names in lowercase.

```
select lower(emp_name) from employeee;
```

-- Q59. Display the length of every employee name.

```
select length(emp_name) from employeee;
```

-- Q60. Display the first three characters of every employee name.

```
select left(emp_name,3) from employeee;
```

-- Q61. Display employee_name and salary rounded to the nearest thousand using an appropriate expression.

```
select emp_name, round(salary,-3) from employeee;
```

-- Q62. Display the current date.

```
select curdate();
```

-- Q63. Calculate how many years each employee has worked using hire_date.

```
select emp_name,hire_date, timestampdiff(year,hire_date , curdate()) from employeee;
```

-- Q64. Replace NULL department values with 'Unassigned' in the query result.

```
select emp_name , coalesce(department,'unassigned') from employeee;
```

-- Q65. Use CASE to label employees as Low, Medium, or High salary based on salary ranges.

```
select emp_name, salary ,  
case  
    when salary < 50000 then 'low'  
    when salary between 50000 and 80000 then 'medium'  
    else 'high'  
end as sal_category  
from employeee;
```

4. Aggregate Functions, GROUP BY & HAVING (66–90)

-- Q66. Find the total number of employees.

```
select count(*) from employeee;
```

-- Q67. Find the total salary paid to all employees.

```
select sum(salary) from employeee;
```

-- Q68. Find the average salary of all employees.

```
select avg(salary) from employeee;
```

-- Q69. Find the highest salary.

```
select max(salary) from employeee;
```

-- Q70. Find the lowest salary.

```
select min(salary) from employeee;
```

-- Q71. Find the number of employees in Hyderabad.

```
select count(*) from employeee  
where emp_city = 'hyderabad';
```

-- Q72. Find the total salary paid to employees in Mumbai.

```
select sum(salary) from employeee  
where emp_city = 'mumbai';
```

-- Q73. Find the average salary of employees in Bangalore.

```
select avg(salary) from employeee  
where emp_city = 'bangalore';
```


-- Q74. Find the number of employees in each department.

```
select department, count(*) from employeee
group by department;
```

-- Q75. Find the average salary in each department.

```
select department , avg(salary) from employeee
group by department;
```

-- Q76. Find the maximum salary in each department.

```
select department , max(salary) from employeee
group by department;
```

-- Q77. Find the minimum salary in each department.

```
select department , min(salary) from employeee
group by department;
```

-- Q78. Find the total salary in each department.

```
select department , sum(salary) from employeee
group by department;
```

-- Q79. Find the number of employees in each city.

```
select count(*), emp_city from employeee
group by emp_city;
```

-- Q80. Find the average salary in each city.

```
select avg(salary), emp_city from employeee
group by emp_city;
```

-- Q81. Find departments having more than 5 employees.

```
select department , count(*) from employeee
group by department
having count(*) > 5;
```

-- Q82. Find departments whose average salary is greater than 60000.

```
select department, avg(salary) from employeee
group by department
having avg(salary) > 60000;
```

-- Q83. Find cities whose total salary is greater than 200000.

```
select emp_city , sum(salary) from employeee
group by emp_city
having sum(salary) > 200000;
```

-- Q84. Find the department with the highest total salary.

```
select department,max(salary) from employeee  
group by department  
order by max(salary) desc  
limit 1;
```

-- Q85. Find the department with the highest average salary.

```
select department,avg(salary) from employeee  
group by department  
order by avg(salary) desc  
limit 1;
```

-- Q86. Display employee count and average salary for each department and city combination.

```
select department,emp_city,count(*),avg(salary) from employeee  
group by department,emp_city;
```

-- Q87. Find the difference between the maximum and minimum salary in each department.

```
select department , max(salary) - min(salary) from employeee  
group by department;
```

-- Q88. Find departments where the maximum salary is greater than 70000.

```
select department , max(salary) from employeee  
group by department  
having max(salary) > 70000;
```

-- Q89. Find cities having at least 3 employees with salary above 60000.

```
select emp_city ,salary from employeee  
where salary > 60000  
order by salary  
limit 3;
```

-- Q90. Find the total number of distinct departments and distinct cities.

```
select count(distinct(department)), count(distinct(emp_city)) from employeee ;
```

5. Constraints, Keys & Data Integrity (91–105)

-- Q91. Create a products table with product_id as PRIMARY KEY.

```
create table product (pro_id int primary key);
```

-- Q92. Create a users table where email must be UNIQUE.

```
create table user(email varchar(199) unique);
```

-- Q93. Create an orders table where order_amount must be greater than 0 using CHECK.

```
create table orders(order_amount int check(order_amount > 0));
```

-- Q94. Create a customers table where customer_name cannot be NULL.

```
create table customer(cust_name varchar(50) not null);
```

-- Q95. Create a table with a DEFAULT status value of 'Active'.

```
create table rr(status varchar(30) default 'Active');
```

-- Q96. Insert a row that uses the DEFAULT value.

```
alter table customer
```

```
add c_id int default 34;
```

-- Q97. Attempt to insert a duplicate PRIMARY KEY and explain which constraint is violated.

```
insert into product values
```

```
(1),
```

```
(1);
```

-- id is primary key so it give error because value of primary key is unique, we cant pass duplicate values.

-- Q98. Attempt to insert a duplicate UNIQUE email and explain which constraint is violated.

```
insert into user values
```

```
('raj@gmail.com'),
```

```
("raj@gmail.com");
```

-- it shows error because in user table we pass the unique key into the email column so cant pass the duplicate values.

-- Q99. Create employees and departments with a FOREIGN KEY relationship.

```
create table emp(
```

```
e_id int primary key,
```

```
sal int,
```

```
dept_id int
```

```
);  
create table dept(  
dept_id int ,  
dept_name varchar(30),  
constraint fk_emp_dept  
foreign key(dept_id)  
references department( dept_id));
```

-- Q100. Insert valid parent and child records respecting the FOREIGN KEY.

```
insert into emp values (12,3900);  
insert into dept values(100,'rr');
```

-- Q101. Attempt to insert an employee with a department_id that does not exist.

```
insert into emp values(10,2000,200);
```

-- Q102. Add a CHECK constraint to allow age only between 18 and 60.

```
alter table emp  
add age int check(age between 18 and 60);
```

-- Q103. Create a composite PRIMARY KEY using two columns.

```
create table rj(  
id int,  
name varchar(10),  
age int,  
primary key(id,age)  
);
```

-- Q104. Create an attendance table where employee_id and attendance_date together form a composite key.

```
create table attendances (  
emp_id int ,  
atd_date varchar(20),  
primary key(emp_id,atd_date)  
);
```

-- Q105. Create a table with a named constraint and then identify its purpose.

```
create table employees (  
employee_id int primary key,  
employee_name varchar(100),  
salary int,  
constraint chk_salary check (salary > 0) );
```

6. JOINS Including Self Join (106–135)

-- Q106. Display employee_name and department_name using an INNER JOIN.

```
select dept_name, emp_name from employeee as e
inner join department as d
on e.dept_id = d.dept_id;
```

-- Q107. Display employee_name, salary, department_name, and location using a JOIN.

```
select emp_name , salary , dept_name , location from employeee as e
inner join department as d
on e.dept_id = d.dept_id;
```

-- Q108. Find employees working in the Data Science department using a JOIN.

```
select * from employeee as e
inner join department as d
on e.dept_id = d.dept_id
where department = 'data science';
```

-- Q109. Find employees working in departments located in Hyderabad.

```
select * from employeee as e
inner join department as d
on e.dept_id = d.dept_id
where location = 'hyderabad';
```

-- Q110. Display all employees even if they do not belong to any department.

```
select * from employeee as e
left join department as d
on e.dept_id = d.dept_id;
```

-- Q111. Display all departments even if they have no employees.

```
select * from department as d
left join employeee as e
on d.dept_id = e.dept_id;
```

-- Q112. Find departments that currently have no employees.

```
select * from department as d
left join employeee as e
on d.dept_id = e.dept_id
where emp_id is null;
```

-- Q113. Find employees who do not have a matching department.

```
select * from employee as e
left join department as d
on e.dept_id = d.dept_id
where d.dept_id is null;
```

-- Q114. Find the number of employees in each department using JOIN and GROUP BY.

```
select department , count(e.emp_name) from employee as e
inner join department as d
on e.dept_id = d.dept_id
group by department;
```

-- Q115. Find the average salary in each department using JOIN and GROUP BY.

```
select department, avg(salary) from employee as e
left join department as d
on e.dept_id = d.dept_id
group by department;
```

-- Q116. Display each department with employee count including departments with zero employees.

```
select department , count(emp_name) from department as d
left join employee as e
on d.dept_id = e.dept_id
group by department;
```

-- Q117. Display each department with total salary including departments with zero employees.

```
select department , sum(salary) from employee as e
right join department as d
on e.dept_id = d.dept_id
group by department;
```

-- Q118. Join departments with projects and display department_name and project_name.

```
select dept_name , project_name from department as d
inner join project as p
on d.dept_id = p.dept_id;
```

-- Q119. Display employees, department names, and project names using appropriate joins.

```
select dept_name , emp_name , project_name from employeee as e
inner join department as d
on e.dept_id = d.dept_id
inner join project as p
on d.dept_id = p.dept_id;
```

-- Q120. Find departments that have at least one project.

```
select distinct(dept_name),project_name from department as d
left join project as p
on d.dept_id = p.dept_id;
```

-- Q121. Find departments that have no projects.

```
select dept_name,project_name from department as d
left join project as p
on d.dept_id = p.dept_id
where project_name is null;
```

-- Q122. Create an employee-manager relationship using manager_id in the employees table.

```
alter table employeee
add manager_id int;
alter table employeee
add constraint fk_manager
foreign key (manager_id)
references employeee(emp_id);
select * from employeee;
insert into employeee values
(7,'rak kiran','data',23400,'rajkn@gmail.com','bbsr','2023-07-11',200,'rajkn@gmail.com',3);
```

-- Q123. Using a SELF JOIN, display employee name and manager name.

```
select e.emp_name as employeee_name , m.emp_name as manager_name from
employeee as e
inner join employeee as m
on e.manager_id = m.emp_id;
```

-- Q124. Using a SELF JOIN, display only employees who have managers.

```
select e.emp_name from employeee as e
inner join employeee as m
on e.manager_id = m.emp_id;
```

-- Q125. Using a SELF JOIN, display employees who do not have managers.

```
select e.emp_name as emp,m.emp_name as manager from employee as e
left join employee as m
on e.manager_id = m.emp_id
where e.manager_id is null;
```

-- Q126. Find all managers who manage at least one employee.

```
select m.emp_name from employee as e
right join employee as m
on e.manager_id = m.emp_id
where e.manager_id is not null;
```

-- Q127. Count the number of employees reporting to each manager.

```
select count(e.emp_name) from employee as e
right join employee as m
on e.manager_id = m.emp_id;
```

-- Q128. Find managers who manage more than 3 employees.

```
select e.emp_name,count(e.emp_name) from employee as e
right join employee as m
on e.manager_id = m.emp_id
group by e.emp_name
having count(e.emp_id) > 3;
```

-- Q129. Display employee, manager, and employee salary using a SELF JOIN.

```
select e.emp_name , m.emp_name , e.salary from employee as e
inner join employee as m
on e.manager_id = m.emp_id;
```

-- Q130. Find employees whose salary is greater than their manager's salary.

```
select e.emp_name , m.emp_name , e.salary from employee as e
inner join employee as m
on e.manager_id = m.emp_id
where e.salary > m.salary;
```

-- Q131. Find employees who work in the same department as their manager.

```
select e.emp_name , m.emp_name , e.salary from employee as e
inner join employee as m
on e.manager_id = m.emp_id
where e.dept_id = m.dept_id;
```


-- Q132. Use a LEFT JOIN to find unmatched rows between employees and departments.

```
select e.emp_id,e.emp_name,e.dept_id from employee e
left join department d
on e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

-- Q133. Use multiple JOINS to display employee, department, and project information.

```
select * from employee as e
inner join department as d
on e.dept_id = d.dept_id
inner join project as p
on d.dept_id = p.dept_id;
```

-- Q134. Find the total project budget for each department.

```
select d.dept_name,sum(p.budget) from department as d
left join project as p
on d.dept_id = p.dept_id
group by d.dept_name;
```

-- Q135. Find departments whose total project budget is greater than their department budget.

```
select d.budget,sum(p.budget) as sum_pro from department as d
inner join project as p
on d.dept_id = p.dept_id
group by d.dept_id
having sum_pro > d.budget;
```

7. Subqueries (136–155)

-- Q136. Find employees whose salary is greater than the overall average salary.

```
select salary , avg(salary) from employee
group by salary
having salary > (select avg(salary) from employee);
```

-- Q137. Find the employee or employees with the highest salary using a subquery.

```
select emp_name , salary from employee
where salary > (select max(salary) from employee);
```

-- Q138. Find the employee or employees with the lowest salary using a subquery.

```
select emp_name , salary from employeee
where salary > (select min(salary) from employeee);
```

-- Q139. Find employees who earn the same salary as Alice.

```
select emp_name , salary from employeee
where salary = (select salary from employeee
where emp_name = 'Alice');
```

-- Q140. Find employees who earn more than Alice.

```
select emp_name , salary from employeee
where salary > (select salary from employeee
where emp_name = 'Alice');
```

-- Q141. Find employees working in the same department as Alice.

```
select emp_name , department from employeee
where department = (select department from employeee
where emp_name = 'Alice');
```

-- Q142. Find employees whose salary is greater than the average salary of their department.

```
select emp_name , salary from employeee as e
where salary > (select avg(salary) from employeee
where dept_id = e.dept_id);
```

-- Q143. Find employees whose salary is less than the average salary of their department.

```
select emp_name , salary from employeee as e
where salary < (select avg(salary) from employeee
where dept_id = e.dept_id);
select * from department;
```

-- Q144. Find the department with the highest average salary using a subquery.

```
select dept_name , avg(budget) as avg_sal from department
group by dept_name
having avg(budget) = (select max(avg_sal) from (select avg(budget) as avg_sal from
department
group by dept_name) as highest_sal);
```

-- Q145. Find departments whose average salary is greater than the company average salary.

```
select dept_name , avg(budget) from department
group by dept_name
having avg(budget) > (select avg(budget) from department);
```

-- Q146. Find employees who work in departments located in Hyderabad using a subquery.

```
select emp_name , department from employeee
where department = (select department from employeee
where department = 'hyderabd');
```

-- Q147. Find employees who do not belong to any existing department using NOT IN or NOT EXISTS.

```
select emp_name from employeee
where department not in (select department from employeee);
```

-- Q148. Find departments that have at least one employee using EXISTS.

```
select dept_name from department
where exists (select * from employeee as e
where dept_id = e.dept_id);
```

-- Q149. Find departments that have no employees using NOT EXISTS.

```
select dept_name from department
where not exists (select * from employeee as e
where dept_id = e.dept_id);
```

-- Q150. Find employees whose salary is greater than every employee in the HR department.

```
select emp_name , salary from employeee
where salary > (select salary from employeee
where department = 'HR');
```

-- Q151. Find employees whose salary is greater than at least one employee in the Finance department.

```
select emp_name , salary from employeee
where salary > any (select emp_name from employeee
where department = 'finance');
```

-- Q152. Find the second-highest distinct salary using a subquery.

```
select max(salary) from employeee
where salary < (select max(salary) from employeee);
```

-- Q153. Find the third-highest distinct salary using a subquery.

```
select max(salary) from employeee
where salary < (select max(salary) from employeee
where salary < (select max(salary) from employeee));
```

-- Q154. Find employees with the second-highest distinct salary.

```
select emp_name , salary from employeee
where salary = (select max(salary) from employeee
where salary < (select max(salary) from employeee));
```

-- Q155. Use a correlated subquery to find employees earning above their department average

```
select emp_name , salary from employeee
where salary > (select avg(salary) from employeee as e
where e.department = department);
```

8. CTEs — Common Table Expressions (156–170)

-- Q156. Create a CTE containing employees whose salary is greater than 60000.

```
with highest_salary as
(select emp_name, salary from employeee
where salary > 60000)
select * from highest_salary;
```

-- Q157. Use a CTE to calculate the company average salary and display employees earning above it.

```
with company_sal as
(select avg(salary) as avg_sal from employeee)
select emp_name , salary from employeee
where salary > (select avg_sal from company_sal);
```

-- Q158. Use a CTE to calculate average salary by department.

```
with avg_sal as
(select department ,avg(salary) from employeee
group by department)
select * from avg_sal;
```

-- Q159. Use a CTE to find departments with average salary greater than 60000.

```
with avg_sal as
(select department, avg(salary) from employeee
group by department
having avg(salary) > 60000)
select * from avg_sal;
```

-- Q160. Create two CTEs: one for department statistics and one for high-paying departments.

```
with dept_stat as
(select department , avg(salary) avg_sal,count(*) from employeee
group by department),
avg_sal as (select department,avg_sal from dept_stat)
select * from avg_sal;
```

-- Q161. Use a CTE to calculate total salary by city and filter cities above a chosen threshold.

```
with total_sal as
(select sum(salary) , emp_city from employeee
group by emp_city
having sum(salary) > 60000)
select * from total_sal;
```

-- Q162. Use a CTE to find the highest salary in each department.

```
with high_sal as
(select department , max(salary) from employeee
group by department)
select * from high_sal;
```

-- Q163. Use a CTE to find employees earning above their department average.

```
with avgs as
(select department , avg(salary) from employeee
group by department)
select * from employeee
where salary > (select avg(salary) from avgs);
```

-- Q164. Use a CTE to rank employees by salary.

```
with ranks as
(select emp_id , emp_name , salary , rank()
over (order by salary desc) from employeee)
select * from ranks;
```

-- Q165. Use a CTE to find the top 2 highest-paid employees.

```
with top_highest as
(select salary , emp_name from employeee
order by salary desc
limit 2)
select * from top_highest;
```

-- Q166. Use a CTE with ROW_NUMBER to find the highest-paid employee in each department.

```
with high_pay as
(select department, salary ,
row_number() over(order by salary desc) as rn
from employeee)
select * from high_pay
where rn = 1;
```

-- Q167. Use a CTE with DENSE_RANK to find the second-highest salary in each department.

```
with high_pay as
(select department, salary ,
dense_rank() over(order by salary desc) as rn
from employeee)
select * from high_pay
where rn = 2;
```

-- Q168. Create a CTE that joins employees and departments, then filter the result.

```
with emp_dept as (
    select e.emp_name,d.dept_name,e.salary from employeee e
    inner join department d
    ON e.dept_id = d.dept_id
)
select * from emp_dept
where salary > 60000;
```

-- Q169. Create a CTE containing departments with employee counts and filter departments with more than 3 employees.

```
with emp as
(select department , count(*) as count_e from employeee
group by department)
select * from emp
where count_e > 3;
```

-- Q170. Create multiple dependent CTEs where the second CTE uses the first CTE.

```
with dept_stats as (
    select department,COUNT(*) as employee_count,avg(salary) as avg_salary from
employeee
    group by department),
high_avg_dept as (
    select department,employee_count,avg_salary from dept_stats
)
select * from high_avg_dept;
```

9. Window Functions (171–180)

-- Q171. Assign a unique row number to all employees ordered by salary descending.

```
select department,salary , row_number() over(
order by salary desc) from employeee;
```

-- Q172. Assign a row number to employees within each department ordered by salary descending.

```
select salary , row_number() over( partition by department
order by salary desc) from employeee;
```

-- Q173. Rank employees by salary using RANK().

```
select emp_name , salary ,rank() over(
order by salary desc) from employeee;
```

-- Q174. Rank employees by salary using DENSE_RANK().

```
select emp_name , salary ,dense_rank() over(
order by salary desc) from employeee;
```

-- Q175. Compare ROW_NUMBER, RANK, and DENSE_RANK on duplicate salary values.

```
select emp_name,salary,  
row_number() over(order by salary desc) as row_num,  
rank() over(order by salary desc) as salary_rank,  
dense_rank() over(order by salary desc) as dense_salary_rank from employeee;
```

-- Q176. Display the company average salary beside every employee using AVG() OVER().

```
select emp_name,salary, avg(salary)  
over() as company_avg_salary  
from employeee;
```

-- Q177. Display the department average salary beside every employee using PARTITION BY.

```
select emp_name,department,salary,avg(salary)  
over(partition by department) as department_avg_salary  
from employeee;
```

-- Q178. Calculate a running total of salary ordered by hire_date.

```
select emp_name,salary,hire_date,sum(salary)  
over(order by hire_date) as running_total  
from employeee;
```

-- Q179. Use LAG() to display the previous employee's salary when ordered by hire_date.

```
select emp_name,salary,hire_date,lag(salary)  
over(order by hire_date) as previous_salary  
from employeee;
```

-- Q180. Use LEAD() to display the next employee's salary when ordered by hire_date.

```
select emp_name,salary,hire_date,lead(salary)  
over(order by hire_date) as previous_salary  
from employeee;
```

10. Hard SQL Challenge Questions (181–200)

-- Q181. Find the top 3 highest-paid employees in every department, including ties if appropriate. Hint: Compare ROW_NUMBER, RANK, and DENSE_RANK before choosing one.

```
with highest_sal as(  
select emp_name,salary,department,row_number()  
over(partition by department
```



```
order by salary desc) as ms from employeee)
select emp_name , department , salary from highest_sal
where ms = 3;
```

**-- Q182. Find the employee whose salary is closest to the average salary of their department. Hint: Calculate
-- department average with a window function, then compare absolute differences.**

```
with emp_avg as (
select emp_name,salary,department,avg(salary)
over(partition by department) as dept_avg from employeee)
select emp_name,salary,department,dept_avg,
abs(salary - dept_avg) from emp_avg
where abs(salary - dept_avg) = (
select min(abs(salary - dept_avg)) from emp_avg
);
```

-- Q183. For every department, find the difference between its highest-paid and second-highest distinct salary. Hint:Use DENSE_RANK in a CTE.

```
with ranked_salary as (select department,salary,dense_rank()
over(partition by department order by salary desc) as salary_rank
from employeee),
dept_salary as (select department,
max(case when salary_rank = 1 then salary end) as highest_salary,
max(case when salary_rank = 2 then salary end) as second_highest_salary from
ranked_salary
group by department)
select department,highest_salary,second_highest_salary,highest_salary -
second_highest_salary as salary_difference
from dept_salary;
```

**-- Q184. Find employees whose salary is higher than their manager and whose department average salary is also
-- above the company average. Hint: Combine SELF JOIN, window/aggregate logic, and a CTE.**

```
with dept_avg as (select emp_name,salary,department,manager_id,
avg(salary) over(partition by department) as dept_avg,
avg(salary) over() as company_avg from employeee)
```

```

select e.emp_name,e.salary,e.department,m.emp_name as
manager_name,e.dept_avg,e.company_avg from dept_avg e
join employeee m
on e.manager_id = m.emp_id
where e.salary > m.salary and e.dept_avg > e.company_avg;

```

-- Q185. Find the first employee hired in each department and the most recently hired employee in each department.Hint: Use two rankings ordered in opposite directions.

```

with ranked_employees as (select emp_name,department,hire_date,row_number()
over(partition by department
order by hire_date) as first_rank,
row_number() over(partition by department
order by hire_date desc) as last_rank from employeee)
select department,
max(case when first_rank = 1 then emp_name end) as first_employee,
max(case when last_rank = 1 then emp_name end) as latest_employee from
ranked_employees
group by department;

```

-- Q186. Find departments where every employee earns more than 50000. Hint: Think about MIN(salary) or NOT EXISTS.

```

select department , min(salary) from employeee
group by department
having min(salary) > 50000;

```

-- Q188. Find the longest consecutive hiring-date gap between employees when ordered by hire_date. Hint: Use LAG() and date difference logic.

```

with date_gap as (select emp_name,hire_date,lag(hire_date)
over(order by hire_date) as previous_hire_date from employeee)
select emp_name,hire_date,previous_hire_date,
datediff(hire_date, previous_hire_date) as gap_days from date_gap
where datediff(hire_date, previous_hire_date) = (
select max(datediff(hire_date, previous_hire_date)) from date_gap
);

```

-- Q189. Find employees whose salary increased relative to the previous employee in their department when ordered by hire_date. Hint: PARTITION BY department with LAG().

```
with previous_salary as (  
select emp_name,department,salary,hire_date,lag(salary)  
over(partition by department  
order by hire_date) as previous_salary from employeee)  
select emp_name,department,salary,previous_salary from previous_salary  
where salary > previous_salary;
```

-- Q191. Find employees who share their salary with another employee in the same department. Hint: GROUP BY department and salary, or use a window COUNT().

```
select emp_name,department,salary from (  
select emp_name,department,salary,  
count(*) over(partition by department, salary) as salary_count  
from employeee) as x  
where salary_count > 1;
```

-- Q192. Find the third-highest distinct salary in each department and return all employees who earn it. Hint: DENSE_RANK plus a CTE.

```
with ranked_salary as (  
select emp_name,department,salary,  
dense_rank() over(order by salary desc) as salary_rank  
from employeee)  
select emp_name,department,salary from ranked_salary  
where salary_rank = 3;
```

-- Q194. Find employees who belong to departments with no projects but earn above their department average. Hint:Combine NOT EXISTS with a window calculation.

```
with emp_avg as  
(select emp_name,department,salary,avg(salary) over(  
partition by department) as dept_avg from employeee)  
select emp_name,department,salary,dept_avg from emp_avg  
where salary > dept_avg and not exists (  
select * from project);
```

-- Q198. Find employees whose salary is greater than the average salary of all employees hired before them. Hint: This requires a running window frame or correlated logic.

```
select e1.emp_name, e1.salary, e1.hire_date from employee e1
where e1.salary > (select avg(e2.salary) from employee e2
where e2.hire_date < e1.hire_date);
```